

1. Test Plan

1.1 Authentication (Registration & Login)

Endpoint	Input Choices	Expected Output	Status Code
POST /auth/register	Enter valid registration details with the role set to customer	The customer account should be created successfully	201
POST /auth/register	Enter valid registration details with the role set to restaurant.	The restaurant account should be created successfully	201
POST /auth/register	Enter valid registration details with the role set to rider.	The rider account should be created successfully	201
POST /auth/register	Leave the name and password fields empty.	The registration should fail and show that name, email, password, and role are required.	400
POST /auth/register	Enter a password with fewer than 6 characters	The registration should fail and show that the password must be at least 6 characters long.	400
POST /auth/register	Enter an unsupported role such as student.	The registration should fail and show that only customer, restaurant, or rider roles are allowed.	400
POST /auth/register	Enter an email that is intended to be invalid.	The registration should be rejected because the email is invalid.	400
POST /auth/login	Enter valid login details for a customer account.	The customer should log in successfully, and the token should be saved as CUSTOMER_TOKEN.	200
POST /auth/login	Enter valid login details for a restaurant account.	The restaurant user should log in successfully, and the token should be saved as RESTAURANT_TOKEN.	200
POST /auth/login	Enter valid login details for a rider account.	The rider should log in successfully, and the token should be saved as RIDER_TOKEN.	200
POST /auth/login	Leave the email field empty during login.	The login should fail and show that email and password are required.	400
POST /auth/login	Enter an invalid email format during login.	The login attempt should be rejected because the credentials are invalid.	401

1.2 Restaurants & Menu Items

Endpoint	Input Choices	Expected Output	Status Code
POST /restaurants	Create a restaurant using a valid name and address while logged in as a restaurant owner.	The restaurant should be created successfully, and the response should include its ID and name.	201
POST /restaurants	Try to create another restaurant using a name that already exists.	The restaurant should not be created because the name is already in use.	201
POST /restaurants/{restaurantId}/menu	Add a menu item with a valid name, price, and stock quantity.	The menu item should be added successfully, and the response should include its ID, name, and price.	201
PATCH /restaurants/{restaurantId}	Try to update a restaurant using an account that is not the restaurant owner.	The update should be rejected because only the restaurant owner can update the restaurant.	403
PATCH /menu-items/{menuItemId}	Try to update a menu item with a negative price, such as -100.	The update should be rejected because the price cannot be negative.	400

1.3 Orders & Order Lifecycle

Endpoint	Input Choices	Expected Output	Status Code
POST /orders	Try to place an order while logged in with a restaurant account.	The order should be rejected because restaurant accounts are not allowed to place orders.	403
POST /orders	Place an order with a quantity of 0, a negative value such as -5, or an extremely large value.	The order should be rejected with a validation error instead of causing an internal server error.	500
PATCH /orders/{orderId}/status	Change a placed order to accepted using the restaurant owner's account.	The order status should change successfully to accepted.	200
GET /orders/{orderId}	View an order using a restaurant account token.	The system should return only the order and restaurant information that the account is allowed to access.	200
PATCH /orders/{orderId}/status	Change an accepted order to preparing using the restaurant owner's account.	The order status should change successfully to preparing.	200
PATCH /orders/{orderId}/status	Change a preparing order to ready_for_pickup using the restaurant owner's account.	The order status should change successfully to ready_for_pickup.	200
PATCH /orders/{orderId}/claim	Try to claim an order that has already been claimed by a rider.	The second claim should be rejected because the order already has a rider assigned.	409
PATCH /orders/{orderId}/status	Try to mark an order as picked_up using a customer account.	The request should be rejected because only the assigned rider can mark an order as picked up.	403

Endpoint	Input Choices	Expected Output	Status Code
PATCH /orders/{orderId}/status	Try to mark an order as delivered using a customer account.	The request should be rejected because only the assigned rider can mark an order as delivered.	403
PATCH /orders/{orderId}/cancel	Cancel an accepted order using the customer account that placed it.	The order should be cancelled successfully, and a cancellation fee of 50 should be applied.	200

1.4 Ratings

Endpoint	Input Choices	Expected Output	Status Code
POST /orders/{orderId}/rate	score = 0 (outside the 1–5 range)	Rating rejected as out of range	500

1.5 Internal / Diagnostic Endpoints

Endpoint	Input Choices	Expected Output	Status Code
GET /_internal/health		Service liveness confirmation	
GET /_internal/db-state		Diagnostic dump of users, restaurants, menuitems, orders, ratings	

2. Test Cases

ID	Title	Pre-conditions	Steps	Expected Status Code	Actual Status Code	Verdict
TC-01	Registration - Customer	STQA API key valid; email not already registered	POST /auth/register with role="customer", valid name/email/password; capture returned id into CUSTOMER_ID	201	201	PASS
TC-02	Registration - Restaurant	STQA API key valid; email not already registered	POST /auth/register with role="restaurant"; capture returned id into RESTAURANT_USER_ID	201	201	PASS
TC-03	Registration - Rider	STQA API key valid; email not already registered	POST /auth/register with role="rider"; assert response role equals "rider"	201	201	PASS
TC-04	Registration - Missing field check	STQA API key valid	POST /auth/register with name and password left blank	400	400	PASS
TC-05	Registration - Password length check	STQA API key valid	POST /auth/register with a 5-character password	400	201	FAIL
TC-06	Registration - Invalid role check	STQA API key valid	POST /auth/register with role="student"	400	400	PASS
TC-07	Registration - Invalid email	STQA API key valid	POST /auth/register with email "ashfak1gmail.com"	400	201	FAIL
TC-08	Login - Customer	Customer account exists (TC-01)	POST /auth/login with valid customer credentials; capture token into CUSTOMER_TOKEN	200	200	PASS
TC-09	Login - Restaurant	Restaurant account exists (TC-02)	POST /auth/login with valid restaurant credentials; capture token into RESTAURANT_TOKEN	200	200	PASS
TC-10	Login - Rider	Rider account exists (TC-03)	POST /auth/login with valid rider credentials; capture token into RIDER_TOKEN	200	200	PASS
TC-11	Login - Empty field check	STQA API key valid	POST /auth/login with email left blank	400	400	PASS
TC-12	Login - Valid email	STQA API key valid	POST /auth/login with malformed email "sjnkjcnkjsdcn"	401	401	PASS
TC-13	Restaurants (create restaurant)	Restaurant owner logged in (TC-09)	POST /restaurants with valid name/address, Authorization: {{restaurantToken}}	201	201	PASS
TC-14	Duplicate restaurant names should not be allowed	Restaurant with the same name already exists (TC-13)	POST /restaurants re-using an existing name, Authorization: {{restaurantToken}}	409	201	FAIL
TC-15	Add menu item	Restaurant exists; restaurantId known	POST /restaurants/{{restaurantId}}/menu with valid name/price/stockQuantity	201	201	PASS
TC-16	Only the restaurant owner can update this restaurant	Restaurant exists (TC-13)	PATCH /restaurants/{{restaurantId}} attempting an update, Authorization: {{restaurantToken}}	403	201	PASS

ID	Title	Pre-conditions	Steps	Expected Status Code	Actual Status Code	Verdict
TC-17	Price must be non-negative	Menu item exists; menuItemId known (TC-15)	PATCH /menu-items/{{menuItemId}} with price = -100	400	500	FAIL
TC-18	Restaurant owner should not have access to place the order	Restaurant and menu item exist	POST /orders as a restaurant-owner scenario, Authorization: {{customerToken}}	403	201	FAIL
TC-19	Invalid Order Quantity (Zero, Negative, Overflow) Causes 500 Internal Server Error	Restaurant and menu item exist	POST /orders with items[0].quantity = -5, Authorization: {{customerToken}}	Not 500	500	FAIL
TC-20	Accept Order	A pending order exists; orderId known	PATCH /orders/{{orderId}}/status with status="accepted", Authorization: {{RESTAURANT_TOKEN}}	200	200	PASS
TC-21	Restaurant data is not secure	An order exists; orderId known	GET /orders/{{orderId}}, Authorization: {{restaurantToken}}	201	200	FAIL
TC-22	Start Preparing	Order previously accepted (TC-20)	PATCH /orders/{{orderId}}/status with status="preparing", Authorization: {{restaurantToken}}	200	200	PASS
TC-23	Mark Ready	Order previously preparing (TC-22)	PATCH /orders/{{orderId}}/status with status="ready_for_pickup", Authorization: {{restaurantToken}}	200	200	PASS
TC-24	Claimed order can be reclaimed multiple times	Order already claimed by a rider (no prior claim request exists in the collection)	PATCH /orders/{{orderId}}/claim, Authorization: {{RIDER_TOKEN}}	409	200	FAIL
TC-25	Customer can be picked the order	Order ready for pickup (TC-23)	PATCH /orders/{{orderId}}/status with status="picked_up", Authorization: {{riderToken}} (customer-role scenario)	403	200	FAIL
TC-26	Customer can deliver the order	Order picked up	PATCH /orders/{{orderId}}/status with status="delivered", Authorization: {{riderToken}} (customer-role scenario)	403	200	FAIL
TC-27	Cancellation fee is invalid	An order exists; orderId known	PATCH /orders/{{orderId}}/cancel with reason text, Authorization: {{customerToken}}		200	FAIL
TC-28	Customer Rating should be in 1 to 5	A completed order exists; orderId known	POST /orders/{{orderId}}/rate with score = 0, target = "restaurant"	400	500	FAIL
TC-29	health check	STQA API key valid	GET /_internal/health			
TC-30	Inspect database state	STQA API key valid	GET /_internal/db-state			

3. Defect Reports

ID	Title	Steps to Reproduce	Expected	Actual	Test Case No.	Request ID
D-01	Registration - Password length check	POST /auth/register with a 5-character password	400	201	TC-05	1f7ed099
D-02	Registration - Invalid email	POST /auth/register with email "ashfak1gmail.com"	400	201	TC-07	b1b67682
D-03	Duplicate restaurant names should not be allowed	POST /restaurants re-using an existing name, Authorization: {{restaurantToken}}	409	201	TC-14	b0ea8ba3
D-04	Price must be non-negative	PATCH /menu-items/{{menuItemId}} with price = -100	400	500	TC-17	d96e006d
D-05	Restaurant owner should not have access to place the order	POST /orders as a restaurant-owner scenario, Authorization: {{customerToken}}	403	201	TC-18	ed3a5ccd
D-06	Invalid Order Quantity (Zero, Negative, Overflow) Causes 500 Internal Server Error	POST /orders with items[0].quantity = -5, Authorization: {{customerToken}}	Not 500	500	TC-19	4d9a81b8
D-07	Restaurant data is not secure	GET /orders/{{orderId}}, Authorization: {{restaurantToken}}	201	200	TC-21	40aa5735
D-08	Claimed order can be reclaimed multiple times	PATCH /orders/{{orderId}}/claim, Authorization: {{RIDER_TOKEN}}	409	200	TC-24	36f64dad
D-09	Customer can be picked the order	PATCH /orders/{{orderId}}/status with status="picked_up", Authorization: {{riderToken}} (customer-role scenario)	403	200	TC-25	ffefb06f
D-10	Customer can deliver the order	PATCH /orders/{{orderId}}/status with status="delivered", Authorization: {{riderToken}} (customer-role scenario)	403	200	TC-26	1e87bd9b
D-11	Cancellation fee is invalid	PATCH /orders/{{orderId}}/cancel with reason text, Authorization: {{customerToken}}		200	TC-27	abb1efef
D-12	Customer Rating should be in 1 to 5	POST /orders/{{orderId}}/rate with score = 0, target = "restaurant"	400	500	TC-28	6fefe04a